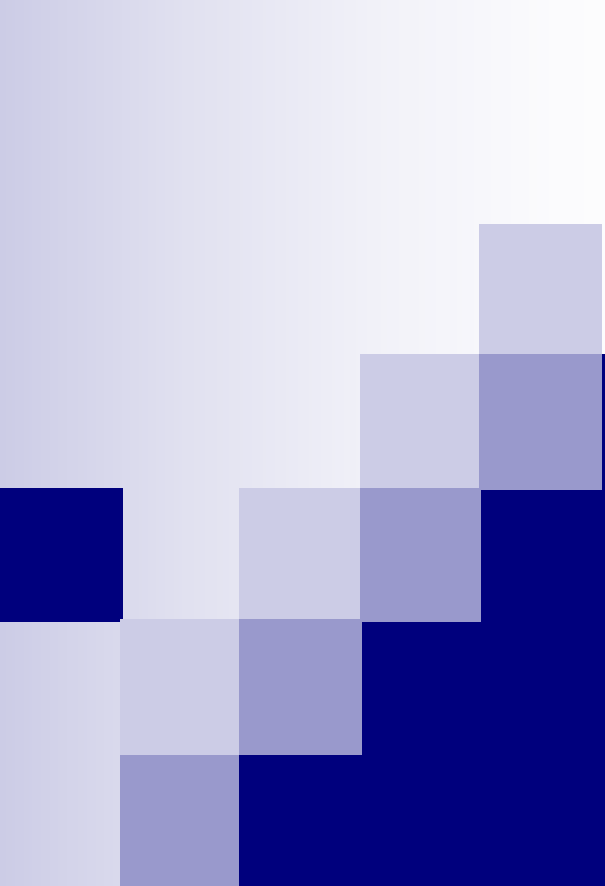




第二章 高级语言及其 语法描述

概述

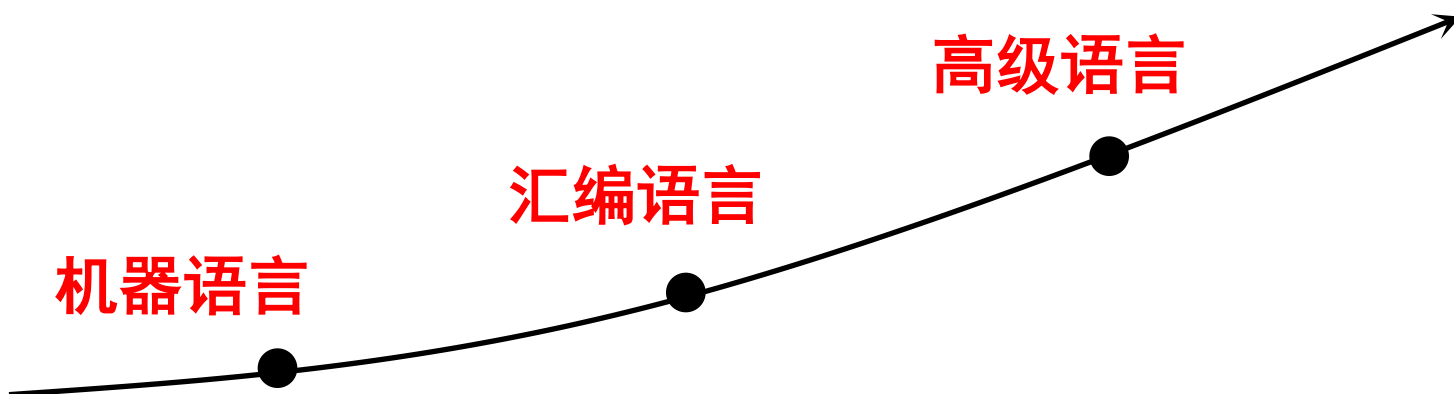
- 要学习和构造编译程序，理解和定义高级语言是必不可少的
- 本章概述高级语言的结构和主要共同特征，重点介绍程序设计语言的语法描述方法

内容线索

- 程序设计语言的定义
- 高级语言的一般特性
- 程序语言的语法描述

程序设计语言的定义

- 程序设计语言是为书写计算机程序而人为设计的符号语言。



各级语言的比较

比较	机器语言	汇编语言	高级语言
硬件识别	是唯一可以识别的语言	不可识别	不可识别
是否可直接执行	可直接执行	不可，需汇编、连接	不可，需编译/解释、连接
特点	✓面向机器 ✓占用内存少 ✓执行速度快 ✓使用不方便	➢面向机器 ➢占用内存少 ➢执行速度快 ➢较为直观 ➢与机器语言一一对应	■面向问题/对象 ■占用内存大 ■执行速度相对慢 ■标准化程度高 ■便于程序交换，使用方便
定位	低级语言，极少使用	低级语言，很少使用	高级语言，种类多，常用

程序语言的内涵

语法

构成语言句子的各个记号之间的组合规律(构成规则)。

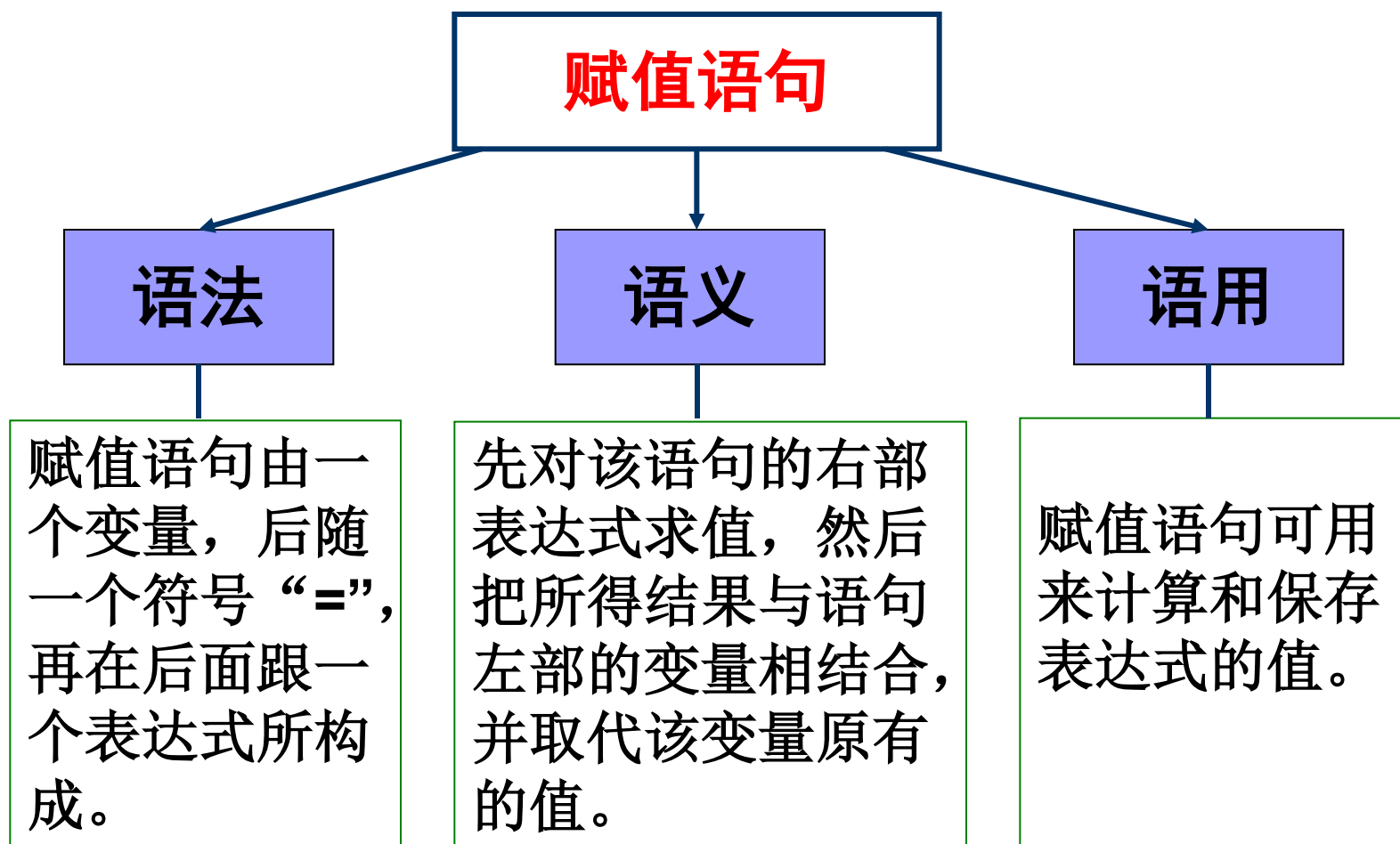
语义

按照各种表示方法所表示的各个记号的特定含义（各个记号和记号所表示的对象之间的关系）。

语用

各个记号或语言词句与其使用之间的关系。

举例：C语言的赋值语句



思考题

- 下面的表述哪个是正确的？（ ）
- A. 标识符是语义概念，名字是语法概念
- B. 标识符是语法概念，名字是语义概念

B

语法

- 语言的语法是指可以形成和产生合式程序的一组规则。它包括词法规则和语法规则。
 - 词法规则是指程序中单词符号的形成规则。单词符号一般包括：标识符、基本字、常量、算符和界符。
 - 语法规则是指程序中语法单位的形成规则。程序语言的语法单位有：表达式、语句、分程序、过程、函数、程序。
- 描述方式：词法规则和语法规则都可以用自然语言、语法图、BNF范式、或文法等描述。

语法描述方式 (1)

(1) 自然语言

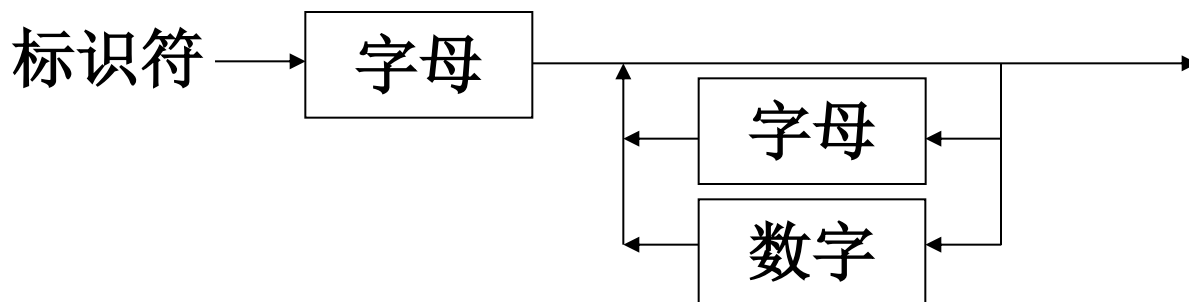
例1. 标识符是由字母后跟若干个（包括0个）字母或数字的符号串组成的。

例2. 赋值语句由一个变量，后随一个符号“=”，再在后面跟一个表达式所构成。

(2) 语法图

□ 是用图解形式描述程序设计语言语法规则的工具。

例1. 标识符的构成规则用语法图描述为：



语法描述方式 (2)

(3) BNF范式

- 巴科斯范式(BNF: Backus-Naur Form 的缩写)是由 John Backus 和 Peter Naur 首先引入的用来描述计算机语言语法的符号集。
- 编程语言几乎都使用巴科斯范式来定义编程语言的语法规则。

简单算术表达式的BNF范式

$\langle \text{简单表达式} \rangle ::= \langle \text{简单表达式} \rangle + \langle \text{简单表达式} \rangle$

$\langle \text{简单表达式} \rangle ::= \langle \text{简单表达式} \rangle * \langle \text{简单表达式} \rangle$

$\langle \text{简单表达式} \rangle ::= (\langle \text{简单表达式} \rangle)$

$\langle \text{简单表达式} \rangle ::= i$

■ 或者

$\langle \text{简单表达式} \rangle ::= \langle \text{简单表达式} \rangle + \langle \text{简单表达式} \rangle$

$\quad | \langle \text{简单表达式} \rangle * \langle \text{简单表达式} \rangle$

$\quad | (\langle \text{简单表达式} \rangle) | i$

附：BNF表示

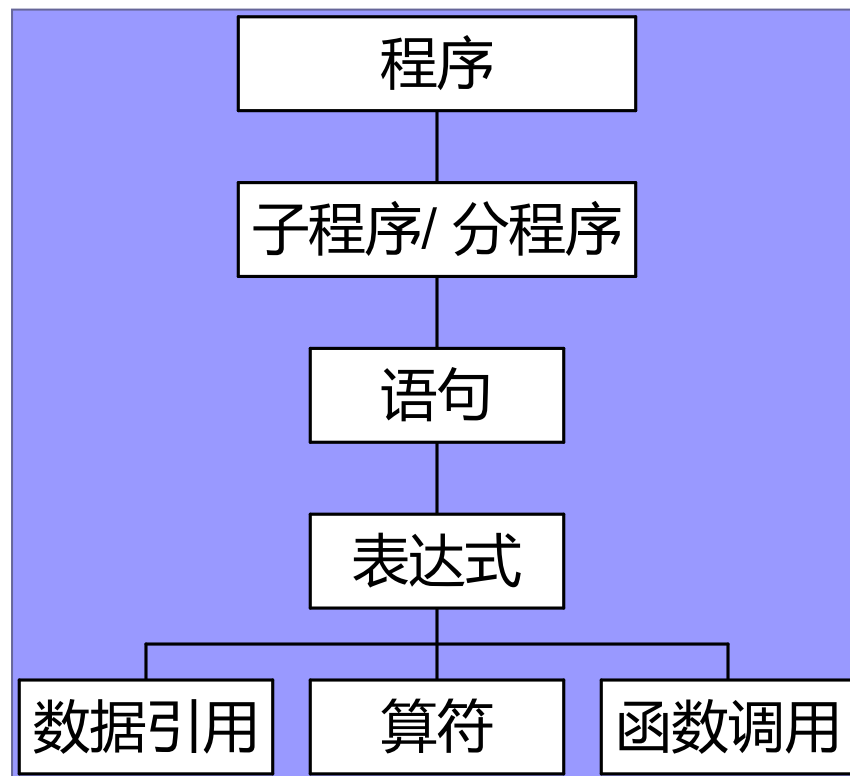
- $\alpha \rightarrow \beta$ 表示为 $\alpha ::= \beta$
- 非终结符用 “<”和 “>”括起来
- 终结符：基本符号集
- 其他
 - $\beta(\alpha_1|\alpha_2\ldots|\alpha_n) \equiv \beta\alpha_1|\beta\alpha_2\ldots|\beta\alpha_n$
 - $\{\alpha_1|\alpha_2\ldots|\alpha_n\}_m^n$
 - $[\alpha] \equiv \alpha | \varepsilon$
 -

语义

- 对于一个语言来说，不仅要给出它的词法、语法规则，而且要定义它的单词符号和语法单位的意义。这就是语义问题。
- 语义是指这样的一组规则，使用它可以定义一个程序的意义。
- 我们采用的方法为：基于属性文法的语法制导翻译方法。

程序语言的功能

- 一个程序语言的基本功能是描述**数据**和**对数据的运算**。
- 所谓程序，从本质上来说是描述一定数据的处理过程。
- 在现今的程序语言中，一个程序大体可以视为层次结构



内容线索

- ✓ 程序设计语言的定义
- 高级语言的一般特性
- 程序语言的语法描述

高级语言的分类

(1) 强制式: **Imperative Language** (一条一条语句地执行)

- 形式: 语句序列
- 举例: **Fortran**、**C**、**Pascal**

(2) 应用式: **Applicative Language** (注重功能, 通过函数描述功能)

- 形式: **func1(…func(n))**
- 举例: **Lisp**

(3) 基于规则: **Rule-Based Language** (检查条件, 满足值则执行动作)

- 形式: **bird(x) :- fly(x) & feather(x)**
- 举例: **Prolog**

(4) 面向对象: **Object-Oriented Language** (封装、继承和多态性)

- 形式: **class**
- 举例: **Smalltalk**、**C++**、**Java**

高级语言的一般特性

- 程序结构
- 数据类型与操作
- 语句与控制结构

程序结构——单层结构

- Fortran程序结构 主程序+若干个辅程序段（子程序、函数）

```
Program Main  
  Read(I,J)  
  Call max(I,J,K)  
  Write(100, K)  
100 Format(...)  
end
```

```
subroutine max(x,y,z)  
  integer x,y,z  
  if x>y then  
    z=x  
  else  
    z =y  
  end
```

程序结构——多层结构

■ Pascal程序结构 程序允许嵌套定义

```
Program P;  
  var a,x:integer;  
  procedure Q(b:integer);  
    var i:integer;  
    procedure R(u:integer;Var v:integer);  
      var c,d:integer;  
      begin  
        ...  
      end  
    begin  
      ...  
    end  
  begin  
    ....  
  end
```

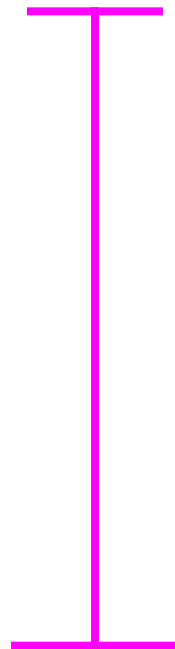
- **作用域**：一个名字能被使用的区域范围称作这个名字的作用域。
- 允许同一个标识符在不同的过程中代表不同的名字。
- 名字作用域规则——“**最近嵌套原则**”
 - 一个在子程序**B1**中说明的名字**X**只在**B1**中有效（局部于**B1**）；
 - 如果**B2**是**B1**的一个内层子程序且**B2**中对标识符**X**没有新的说明，则原来的名字**X**在**B2**中仍然有效。如果**B2**对**X**重新作了说明，那么，**B2**对**X**的任何引用都是指重新说明过的这个**X**。

```

program main
  var A, B : real;
  ...
  procedure P1
    var B:boolean;
    ...
  begin
    ...
  end
  procedure P2
    var A:integer;
    ...
  begin
    ...
  end
begin
  ...
end

```

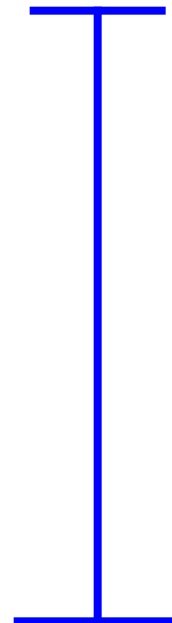
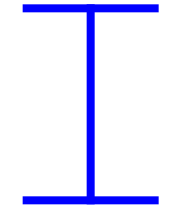
A(real)



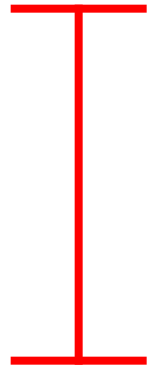
A(integer)



B(real)



B(bool)



初等类型、复合类型到抽象数据类型

- 类型本不存在
 - 内存里存储的内容，你认为它是什么，它就是什么
- 高级语言设计了初等数据类型：整型、浮点型、字符型等。不同的语言也会定义不同的初等类型等。
 - 初等数据类型并不能方便地解决所有问题
- 复合数据类型是初等数据类型迭代派生而来
 - 典型的代表就是“结构”，数组也可算作此类
- 抽象数据类型 (ADT)在复合数据类型的基础上增加了对数据的操作
 - 抽象数据类型进而进化为“类(Class)”

类型

- 每个被计算对象都带有自己的类型，以类型作为值的属性的概括，因此每个类型都意味着一个值的集合。
- 不同类型的值具有不同的操作运算
- 类型是一个值的集合和定义在这个值集上的一组操作的总称。
 - 如C语言中的整型变量(int)，其值集为某个区间上的整数，定义在其上的操作为+, -, *, /等

数据类型

- 数据类型通常包括以下三种要素：
 - a. 用于区别这种类型的数据对象的属性
 - b. 这种类型的数据对象可以具有的值
 - c. 可以作用于这种类型数据对象的操作

抽象数据类型

- 一个抽象数据类型（**Abstract Data Type, ADT**）定义为：
 - （1）一个数据对象集，数据对象由一个或多个类型定义；
 - （2）一个作用于这些数据对象的抽象操作集；
 - （3）完全封装，用户除了能使用该类型的操作来处理这类数据对象之外，不能作其他的处理。
- 抽象数据类型有两个重要特征：**信息隐蔽**和**数据封装**，使用与实现相分离

抽象数据类型

对外操作接口
结构数据维护接口

Queue (ADT)

InQueue

OutQueue

IsEmpty

Overflow



First Come First Service (queue)

结构数据的
物理实现

抽象数据类型（续）

对外操作接口
结构数据维护接口

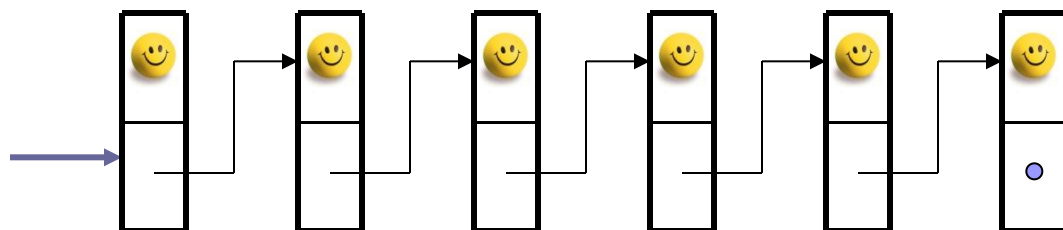
Queue (ADT)

InQueue

OutQueue

IsEmpty

Overflow



First Come First Service (queue)

结构数据的
物理实现

抽象数据类型的特点

- 数据抽象用**ADT**描述程序处理的实体时，强调的是其本质的特征、其所能完成的功能以及它和外部用户的接口（即外界使用它的方法）。
- 数据封装将实体的外部特性和其内部实现细节分离，并且对外部用户隐藏其内部实现细节。

数据抽象的优点

- 程序组织和修改
- 可读性、可靠性、可维护性
 - 通过隐藏数据表示，用户代码不能直接访问该类型对象，不依赖于其表示，因此可以修改该类型对象的表示而不影响用户代码

语句与控制结构

- 表达式

- 语句

- 简单语句：不含其它语句成分的基本句。

- 说明语句

- 赋值语句

- 控制语句

- 过程调用语句

- 复合语句：句中有句的语句

表达式

- 表达式：一个表达式是由运算量（亦称操作数，即数据引用或函数调用）和算符组成的。
- 对于大多数程序语言来说，表达式的形成规则可概括为：
 - （1）变量（包括下标变量）、常数是表达式；
 - （2）若 $E1$ 、 $E2$ 为表达式， θ 为二元算符，则 $E1 \theta E2$ 为表达式；
 - （3）若 E 为表达式， θ 为一元算符，则 θE 为表达式；
 - （4）若 E 为表达式，则 (E) 是表达式。

语句

- 不同程序语言含有不同形式和功能的各种语句。
- 从功能上说语句大体可分**执行性语句**和**说明性语句**两大类：
 - 说明性语句旨在定义不同数据类型的变量或运算。
 - 执行性语句旨在描述程序的动作。
 - 执行性语句又可分为赋值语句、控制语句和输入/输出语句
- 从形式上说，语句可分为**简单句**、**复合句**和**分程序**等。

赋值语句

A := B

- 意义是：“把**B**的值送入**A**所代表的单元”
 - 在赋值句中，赋值号 ‘:=’ 左右两边的变量名扮演着两种不同的角色。对赋值号右边的**B**我们需要的是它的值；对于左边的**A**我们需要的是它们所代表的存储单元（的地址）。
- 为了区分一个名字的这两种特征，我们把一个名字所代表的那个存储单元（地址）称为该名字的**左值**；把一个名字的值称为该名字的**右值**。

控制语句

■ 多数语言中所含的控制语句有：

- 无条件转移语句： **goto L**

- 条件语句： **if B then S**

if B then S1 else S2

- 循环语句： **while B do S**

repeat S until B

for i:=E1 step E2 until E3 do S

- 过程调用语句： **call P(X1,X2,⋯,Xn)**

- 返回语句： **return(E)**

说明语句

- 说明语句旨在定义名字的性质。
- 编译程序把这些性质登记在符号表中，并检查程序中名字的引用和说明是否相一致。

内容线索

- ✓ 程序设计语言的定义
- ✓ 高级语言的一般特性
- 程序语言的语法描述

概述

- 对于高级程序语言及编译程序而言，语言的语法定义是非常重要的。本节将介绍语法结构的形式描述问题。

编译原理=形式语言理论+编译技术

形式语言

■ 自然语言

- 人们平时说话时所使用的一种语言，不同的国家和民族有着不同的语言。

■ 形式语言

- 通过人们公认的符号、表达方式所描述的一种语言，是一种通用语言，没有国籍之分。
- 形式语言是某个字母表上的字符串的集合，有一定的描述范围。

为什么用形式语言

- 形式语言的最初起因：语言学家乔姆斯基（**Chomsky**）想用一套形式化方法来描述语言。
- 形式语言在自然语言研究中起步，在计算机科学中得到广泛应用。
 - 最初的应用：编译
 - 现在已广泛应用在人工智能、图象处理、通信协议、通信软件等多个领域
 - 在计算机理论科学方面：
 - 是可计算理论（算法—在有限步骤内求得解、算法复杂性、停机问题）、定理自动证明、程序转换（程序自动生成）、模式识别等的基础。

形式语言与自动机理论的发展

- 1956年，乔姆斯基（Chomsky）从产生的角度研究语言
 - 文法
- 1951-1956年间，克林（Kleene）从识别的角度研究语言
 - 自动机
- 1959年，乔姆斯基不仅确定了文法和自动机分别从生成和识别的角度去表达语言，而且证明了文法与自动机的等价性。

形式语言真正诞生

基本概念

- **字母表**：元素的非空有穷集合。

例： $\{a, b, c, \dots, z\}$ （拉丁字母表）

$\{\alpha, \beta, \gamma, \dots, \omega\}$ （希腊字母表）

$\{0, 1\}$ （二进制数字字母表）

- **符号**：字母表中的元素。如 $a, b, \dots$

基本概念

- **符号串**：字母表中的符号组成的任何有穷序列。
例. 设字母表 $\Sigma = \{a, b, c\}$
其符号串有： $a, b, c, ab, ac, aa, abc, \dots$
 - 符号串与符号组成的顺序有关。
- **符号串的长度**：符号串中符号的数目, 用 $|x|$ 表示
- **空符号串**（空字）：不包含任何符号的符号串，记为 ϵ 。

子符号串

- 设有非空符号串 $u = xvy$ ，则称 v 为符号串 u 的子符号串。

例. 符号串 $x = a + b - (c + d)$

则 a , $a + b -$, $(c + d)$ 等都是 x 的子符号串，

且其长度分别为： $|a| = 1$,

$$|a + b -| = 4,$$

$$|(c + d)| = 5$$



字符串的前缀与后缀

■ 字符串的前缀与后缀

- 字符串左部的任意子串，称为字符串的前缀；
- 字符串右部的任意子串，称为字符串的后缀。

例. 字母表 $A=\{a,b,c\}$ 上的字符串 $x=ab$, 则 x 的

前缀有: ε 、 a 、 ab ;

后缀有: ε 、 b 、 ab 。



基本概念

- **符号串集合**：若集合中所有元素都是某字母表 Σ 上的符号串，则称之为该字母表上的符号串集合。
 - 用 Σ^* 表示 Σ 上的所有符号串的全体，空字也包括在其中。
 - 例. 若 $\Sigma=\{a,b\}$ 则 $\Sigma^*=\{\varepsilon,a,b,aa,ab,bb,aaa,\dots\}$ 。
 - ϕ 表示不含任何元素的**空集** $\{\}$
- **注意区分**： ϕ 、 ε 、 $\{\varepsilon\}$

(连接) 积

- Σ^* 的子集 U 和 V 中的 (连接) 积定义为:

$$UV = \{\alpha\beta \mid \alpha \in U \ \& \ \beta \in V\}$$

□ 即集合 UV 中的符号串是由 U 和 V 的符号串连接而成的。

例. 设 $U = \{a, b\}$, $V = \{\alpha, \beta, \gamma\}$

则 $UV = \{a\alpha, a\beta, a\gamma, b\alpha, b\beta, b\gamma\}$

- 注意: 一般 $UV \neq VU$, 但 $(UV)W = U(VW)$.



n次（连接）积

- V自身的n次（连接）积记为：

$$V^n = \underbrace{VVV\dots V}_n$$

- 规定 $V^0 = \{\varepsilon\}$.
- 令： $V^* = V^0 \cup V^1 \cup V^2 \cup \dots$ ，称 V^* 是V的闭包。
- 闭包 V^* 中的每个符号都是由V中的符号串经有限次连接而成的。
- 记 $V^+ = VV^*$ ，称 V^+ 是V的正则闭包。



闭包

■ **定理.** 若 A 是符号串集合, 则 $A^*=(A^*)^*$ 。

证明: 显然 $A^* \subseteq (A^*)^*$, 现证 $(A^*)^* \subseteq A^*$

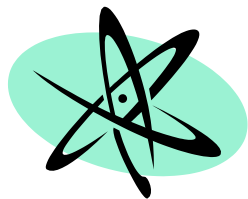
任给符号串 $x \in (A^*)^*$, 则存在 n , 使得 $x \in (A^*)^n$, x 必存在 n 个子串 $x_1, \dots, x_n$, 使得 $x = x_1 \dots x_n$, 且 $x_i \in A^*$ ($i=1, \dots, n$)。由 $x_i \in A^*$, 必存在整数 p_i , 使得 $x_i \in A^{p_i}$,

$$\text{令 } M = \sum_{i=1}^n p_i$$

则 $x \in A^M$, 因此推得 $x \in A^*$,

所以 $(A^*)^* \subseteq A^*$,

故 $A^*=(A^*)^*$ 。



文法 (Grammar)

- 所谓文法是用来定义语言的一个数学模型。
- 表示语言的方法：
 - 若语言 L 是有限集合，可用列举法
 - 若 L 是无限集合（集合中的每个元素有限长度），用其他方法。
 - 方法一：文法产生系统，由定义的文法规则产生出语言的每个句子
 - 方法二：机器识别系统：当一个字符串能被一个语言的识别系统接受，则这个字符串是该语言的一个句子，否则不属于该语言。

Chomsky 文法体系

■ Chomsky文法体系中，任何一种文法必须包含

- 两个不同的有限符号的集合
 - 非终结符集合 V_N
 - 终结符集合 V_T
- 一个起始符 S
- 一个形式规则的有限集合 $\mathcal{F}$ （产生式集合）。

■ 注意

- $\mathcal{F}$ 中的产生式是用来产生语言句子的规则，而句子则是仅由终结符组成的字符串。
- 这些字符串必须从一个起始符 S 开始，不断使用 $\mathcal{F}$ 中的产生式而导出来。
- 文法的核心是产生式的集合，它决定了语言中句子的产生。

文法的形式定义

■ 文法**G**是一个四元组 **$G=(V_N, V_T, S, \mathcal{P})$** ，其中

V_N ：非终结符的有限集合

V_T ：终结符的有限集合， **$V_N \cap V_T = \Phi$**

S：开始符号且 **$S \in V_N$** 。

$\mathcal{P}$ ：形式为 **$P \rightarrow \alpha$** 的产生式的有限集合，

且 **$P \in (V_N \cup V_T)^* V_N (V_N \cup V_T)^*$** ， **$\alpha \in (V_N \cup V_T)^*$**

文法示例

- 文法 $G_1 = (\{N\}, \{0, 1\}, \{N \rightarrow 0N, N \rightarrow 1N, N \rightarrow 0, N \rightarrow 1\}, N)$

其中: $V_N = \{N\}$,

$V_T = \{0, 1\}$,

$S = N$ 。

$\mathcal{F} = \{N \rightarrow 0N, N \rightarrow 1N, N \rightarrow 0, N \rightarrow 1\}$,

文法的解释 (1)

- **非终结符号**：需要进一步定义的符号，不会出现在程序中。
 - 用来代表语法范畴。如“算术表达式”、“布尔表达式”、“过程”等。
 - 一个非终结符代表一个一定的语法概念，因此非终结符是一个类（或集合）记号，而不是个体记号。
- **终结符号**：不需要再定义，会出现在程序中。
 - 组成语言的基本符号，即在程序语言中的单词符号，如标识符，常数，算符和界符等
- **开始符号S**至少必须在某个产生式的左部出现一次。

文法的解释 (1)

■ 产生式（规则）：定义语法单位的一种书写规则

- 它的形式为： $P \rightarrow \alpha$ （或 $P ::= \alpha$ ）

其中“ $\rightarrow$ ”读作“定义为”， P ， α 为符号串，箭头左边称为产生式左部，箭头右边称为产生式右部。

例： $S \rightarrow 0S1$ ， $0S \rightarrow 01$

- 若干个左部相同的产生式如 $P \rightarrow \alpha_1$ ， $P \rightarrow \alpha_2$ ， $P \rightarrow \alpha_3$ ，...， $P \rightarrow \alpha_n$ 可合并成一个，缩写为

$$P \rightarrow \alpha_1 | \alpha_2 | \alpha_3 | \dots | \alpha_n ,$$

其中,每个 α_i 称为是 P 的一个候选式.

习惯写法

- V_N : 大写字母 A、B、C、S 等
- V_T : 小写字母, 0~9, +、- 等运算符
- α 、 β 、 γ : 文法符号串, $\in (V_T \cup V_N)^*$
- S : 开始符号, 第一个产生式中出现
- $\rightarrow$: 定义符号 (推出)
- $|$: 或

文法的约定

■ 为了简化，文法表示

- 只写出产生式部分
- 约定第一个产生式的左部符号为初始符号

或

在产生式前写上” **G[A]** “ 其中**G**为文法名，**A**为初始符号.

例. 文法**G[N]**: $N \rightarrow 0N, N \rightarrow 1N, N \rightarrow 0, N \rightarrow 1$

文法**G[E]**: $E \rightarrow E + E \mid E * E \mid (E) \mid i$

如何由文法产生语言的句子？

- **基本思想**：从识别符号开始，把当前产生的符号串中的**非终结符号**替换为相应产生式右部的**符号串**，直到最终全由**终结符号**组成。这种替换过程称为推导或产生句子的过程，每一步称为**直接推导**或**直接产生**。

直接推导与归约

■ 直接推导

- 如果 $A \rightarrow \gamma$ 是一个产生式，而 $\alpha, \beta \in (V_T \cup V_N)^*$ ，则将产生式 $A \rightarrow \gamma$ 用于符号串 $\alpha A \beta$ 得到符号串 $\alpha \gamma \beta$ ，记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$ ，称 $\alpha A \beta$ 直接推出 $\alpha \gamma \beta$ 。

■ 归约是推导的逆过程

- 若存在 $\alpha A \beta \Rightarrow \alpha \gamma \beta$ ，则称 $\alpha \gamma \beta$ 能够直接归约成 $\alpha A \beta$ 。

推导

- 推导：设 $\alpha_1, \alpha_2, \dots, \alpha_n$ ($n > 0$) $\in (V_T \cup V_N)^*$ ，且有

$$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$$

则称这个序列是从 α_1 到 α_n 的一个推导。

- 若存在一个 α_1 到 α_n 的推导，则称 α_1 可推导出 α_n 。

- $\alpha_1 \Rightarrow^+ \alpha_n$ (经一步或若干步推导)

- $\alpha_1 \Rightarrow^* \alpha_n$ (经0步或若干步推导)

最左（右）推导（规约）

- 若在推导关系中，**每次**最先替换最左（右）的非终结符，则称为**最左（右）推导**；
- 若在归约过程中，**每次**最先归约最左（右）的非终结符，则称为**最左（右）归约**。

- 例. 文法G[S]: $S \rightarrow AB$ $A \rightarrow A0|1B$ $B \rightarrow 0|S1$,
请给出句子101001的最左和最右推导。

最左推导:

$S \Rightarrow AB$
 $\Rightarrow 1BB$
 $\Rightarrow 10B$
 $\Rightarrow 10S1$
 $\Rightarrow 10AB1$
 $\Rightarrow 101BB1$
 $\Rightarrow 1010B1$
 $\Rightarrow 101001$

最右推导:

$S \Rightarrow AB$
 $\Rightarrow AS1$
 $\Rightarrow AAB1$
 $\Rightarrow AA01$
 $\Rightarrow A1B01$
 $\Rightarrow A1001$
 $\Rightarrow 1B1001$
 $\Rightarrow 101001$

句型、句子和语言

- **句型**：假定 G 是一个文法， E 是它的开始符号，如果 $E \xRightarrow{*} \alpha$ ，则称 α 是文法 G 的一个句型。

例 $(E+E)$, $(i+E)$, $(i+i)$, E 都是 $G[E]$ 的句型。

- **句子**：仅由终结符组成的句型称为句子。

例 $(i \times i + i)$, $(i + i)$ 都是 $G[E]$ 的句子。

- **语言**：文法 G 所产生句子的全体，即：

$$L(G) = \{\alpha \mid S \xRightarrow{+} \alpha, \alpha \in V_T^*\}$$

例. 设有文法**G1[S]**: $S \rightarrow bA$, $A \rightarrow aA \mid a$

试求此文法所描述的语言。

解： 因为从开始符号**S**出发可推出下列句子：

$$S \Rightarrow bA \Rightarrow ba$$

$$S \Rightarrow bA \Rightarrow baA \Rightarrow baa$$

$$S \Rightarrow bA \Rightarrow baA \Rightarrow baaA \Rightarrow baaa$$

...

$$S \Rightarrow bA \Rightarrow baA \Rightarrow \dots \Rightarrow baa\dots a$$

所以, $L(G1) = \{ ba^n \mid n > 1 \}$

例. 设文法 $G_2[S]$: $S \rightarrow AB$
 $A \rightarrow aA|a$
 $B \rightarrow bB|b$

该文法所描述的语言是什么?

解: 从文法开始符号 S 出发, 可推出下列句子:

$$S \Rightarrow AB \Rightarrow ab$$

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aab$$

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aabbB \Rightarrow aabbb$$

$$S \Rightarrow AB \Rightarrow \underbrace{aa \dots a}_m \underbrace{bb \dots b}_n$$

所以, $L(G_2) = \{a^m b^n \mid m, n \geq 1\}$

例. 试对如下语言 $L(G3) = \{a^n b^n \mid n \geq 1\}$ 构造文法 $G3$ 。

解: $L(G3)$ 由以下一些符号串 x 组成:

$n=1, x=ab$

$n=2, x=aabb$

$n=3, x=aaabbb$

...

$L(G3) = \{ab, aabb, aaabbb, \dots\}$

由此可见, 集合 $L(G3)$ 有如下特点:

- 每个符号串呈对称形式, 即 a, b 成对出现;
- $L(G3)$ 为无穷集合, 描述它的规则中含递归定义;
- 文法中终结符只有 a, b 。

因此可用以下两条产生式定义语言 $L(G3)$

$$S \rightarrow aSb \mid ab$$

即: $G3 = (\{S\}, \{a, b\}, \{S \rightarrow aSb \mid ab\}, S)$

例. 文法 $G[<\text{数}>]$:

$<\text{数}> \rightarrow <\text{数字串}>$

$<\text{数字串}> \rightarrow <\text{数字串}><\text{数字}> \mid <\text{数字}>$

$<\text{数字}> \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

语言 $L(G)$ 为非负整数。



例. 写一文法**G3**，使其描述的语言为正奇数集合。

令 $G3 = \{ V_N, V_T, P, \langle \text{正奇数} \rangle \}$ ，其中，

$V_T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$,

$V_N = \{ \langle \text{正奇数} \rangle, \langle \text{一位奇数} \rangle, \langle \text{一位偶数} \rangle, \langle \text{数字串} \rangle, \langle \text{数字} \rangle \}$

S: $\langle \text{正奇数} \rangle$

P: $\langle \text{正奇数} \rangle \rightarrow \langle \text{一位奇数} \rangle \mid \langle \text{数字串} \rangle \langle \text{一位奇数} \rangle$

$\langle \text{数字串} \rangle \rightarrow \langle \text{数字串} \rangle \langle \text{数字} \rangle \mid \langle \text{数字} \rangle$

$\langle \text{数字} \rangle \rightarrow \langle \text{一位奇数} \rangle \mid \langle \text{一位偶数} \rangle$

$\langle \text{一位奇数} \rangle \rightarrow 1 \mid 3 \mid 5 \mid 7 \mid 9$

$\langle \text{一位偶数} \rangle \rightarrow 0 \mid 2 \mid 4 \mid 6 \mid 8$

$\langle \text{正奇数} \rangle \rightarrow \langle \text{一位奇数} \rangle \mid \langle \text{数字串} \rangle \langle \text{一位奇数} \rangle$

$\langle \text{数字串} \rangle \rightarrow \langle \text{数字串} \rangle \langle \text{数字} \rangle \mid \langle \text{数字1} \rangle$

$\langle \text{数字} \rangle \rightarrow \langle \text{一位奇数} \rangle \mid \langle \text{一位偶数} \rangle$

$\langle \text{数字1} \rangle \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$\langle \text{一位奇数} \rangle \rightarrow 1 \mid 3 \mid 5 \mid 7 \mid 9$

$\langle \text{一位偶数} \rangle \rightarrow 0 \mid 2 \mid 4 \mid 6 \mid 8$

或者:

$\langle \text{正奇数} \rangle \mathbf{S}$, $\langle \text{一位奇数} \rangle \mathbf{A}$, $\langle \text{一位偶数} \rangle \mathbf{E}$, $\langle \text{数字串} \rangle \mathbf{B}$,

$\langle \text{数字} \rangle \mathbf{C}$, $\langle \text{数字1} \rangle \mathbf{D}$.

$\mathbf{S} \rightarrow \mathbf{A} \mid \mathbf{BA}$

$\mathbf{B} \rightarrow \mathbf{BC} \mid \mathbf{D}$

$\mathbf{C} \rightarrow 0 \mid \mathbf{E} \mid \mathbf{A}$

$\mathbf{D} \rightarrow \mathbf{E} \mid \mathbf{A}$

$\mathbf{E} \rightarrow 2 \mid 4 \mid 6 \mid 8$

$\mathbf{A} \rightarrow 1 \mid 3 \mid 5 \mid 7 \mid 9$

同一句型有不同的推导序列

- 设有文法 $G[N_1]: N_1 \rightarrow N \quad N \rightarrow ND|D$
 $D \rightarrow 0|1|2$

则句子12可由若干种不同的推导序列推导出来:

(1) $N_1 \Rightarrow N \Rightarrow ND \Rightarrow N2 \Rightarrow D2 \Rightarrow 12$

(2) $N_1 \Rightarrow N \Rightarrow ND \Rightarrow DD \Rightarrow 1D \Rightarrow 12$

- 可见，同一句型（句子）可以通过不同的推导序列推导出来。

Chomsky 文法体系分类

- 文法 $G=(V_N, V_T, S, \mathcal{P})$, $\mathcal{P} : P \rightarrow \alpha$, 其中
 $P \in (V_N \cup V_T)^* V_N (V_N \cup V_T)^*$, $\alpha \in (V_N \cup V_T)^*$ 属于
Chomsky 文法体系
- 该体系对产生式的形式做了一些规定, 分为四类, 即0型、1型、2型、3型文法

3型文法

■ 也称正规文法

□ 右线性文法（**Right-linear Grammar**）：

$A \rightarrow \omega B$ 或 $A \rightarrow \omega$ ，其中 $A、B \in V_N, \omega \in V_T^*$ 。

□ 左线性文法（**Left-linear Grammar**）：

$A \rightarrow B\omega$ 或 $A \rightarrow \omega$ ，其中 $A、B \in V_N, \omega \in V_T^*$ 。

□ 对应的语言：正规语言

□ 对应的自动机：有限自动机（**Finite Automaton**）。

■ 例. 文法 $S \rightarrow aS, S \rightarrow a$

对应正规式： a^+ ，或者 a^*a

2型文法

- 也称上下文无关文法 (**CFG: Context-free Grammar**)
- 产生式的形式为 $P \rightarrow \alpha$, 其中 $P \in V_N$, 且 $\alpha \in (V_N \cup V_T)^*$
 - 对应的语言: 上下文无关语言 (**CFL: Context-free Language**)
 - 对应的自动机: 下推自动机 (**PDA: Pushdown Automaton**)。

2型文法示例

- 例. 上下文无关文法:

$$S \rightarrow 01$$

$$S \rightarrow 0S1$$

产生的语言是 $L = \{ 0^n 1^n \mid n \geq 1 \}$,

如 $0011, 000111, 01 \in L$, 而 $10, 1001, \varepsilon, 010 \notin L$.

但没有任何有限自动机能够接受语言 L .

1 型文法

- 也称上下文有关文法（**CSG: Context-sensitive Grammar**）

- 产生式的形式为 $P \rightarrow \alpha$, 其中

$$|P| \leq |\alpha|, \quad \alpha \in (V_N \cup V_T)^*, \quad P \in (V_N \cup V_T)^* V_N (V_N \cup V_T)^*$$

- 对应的语言：上下文有关语言（**CSL: Context-sensitive Language**）
- 若不考虑 ϵ , 与线性有界自动机（**LBA, Linear Bounded Automaton**）等价。

1型文法示例

- 例. 下列文法是1型文法

$S \rightarrow aBC | aSBC$

$CB \rightarrow BC$

$aB \rightarrow ab$

$bB \rightarrow bb$

$bC \rightarrow bc$

$cC \rightarrow cc$

0型文法

- 0型文法：无限制文法，短语文法
 - 对应的语言：递归可枚举语言
 - 与图灵机等价。
- 例. 下列文法是0型文法

$S \rightarrow aBC|aSBC$

$CB \rightarrow BC$

$aB \rightarrow ab$

$bB \rightarrow bb$

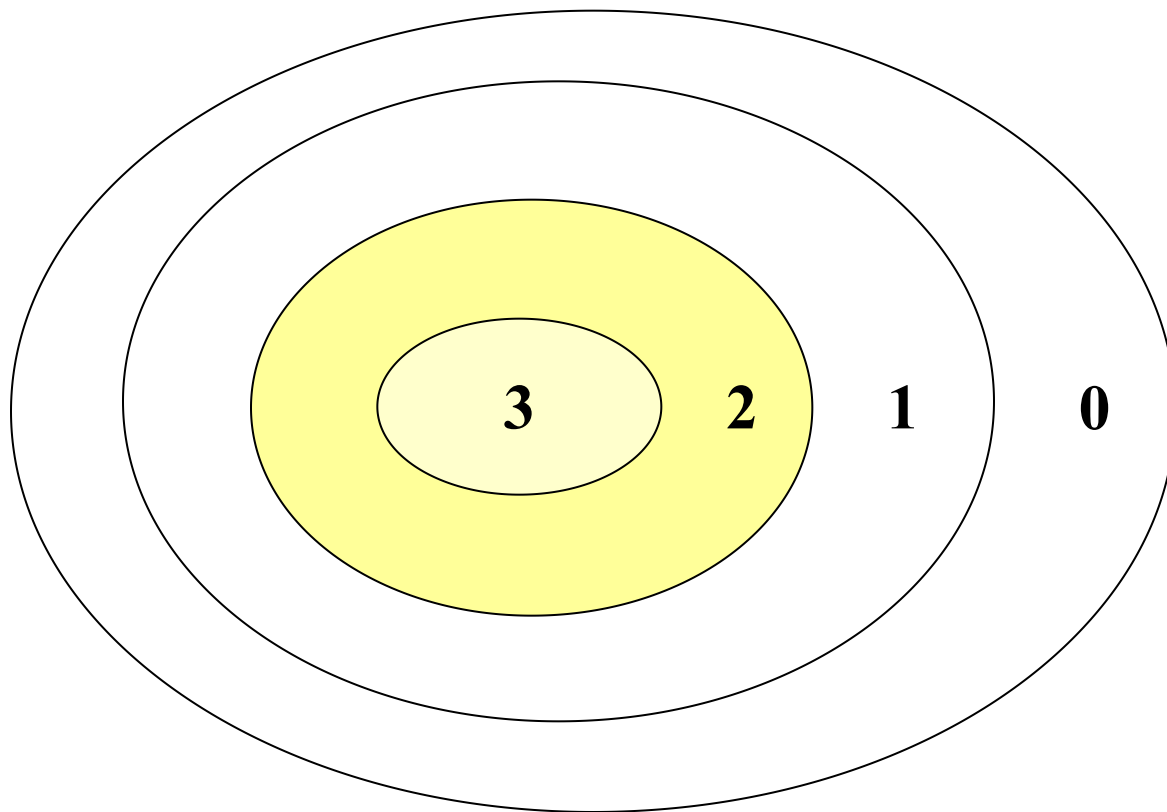
$bB \rightarrow b$

$bC \rightarrow bc$

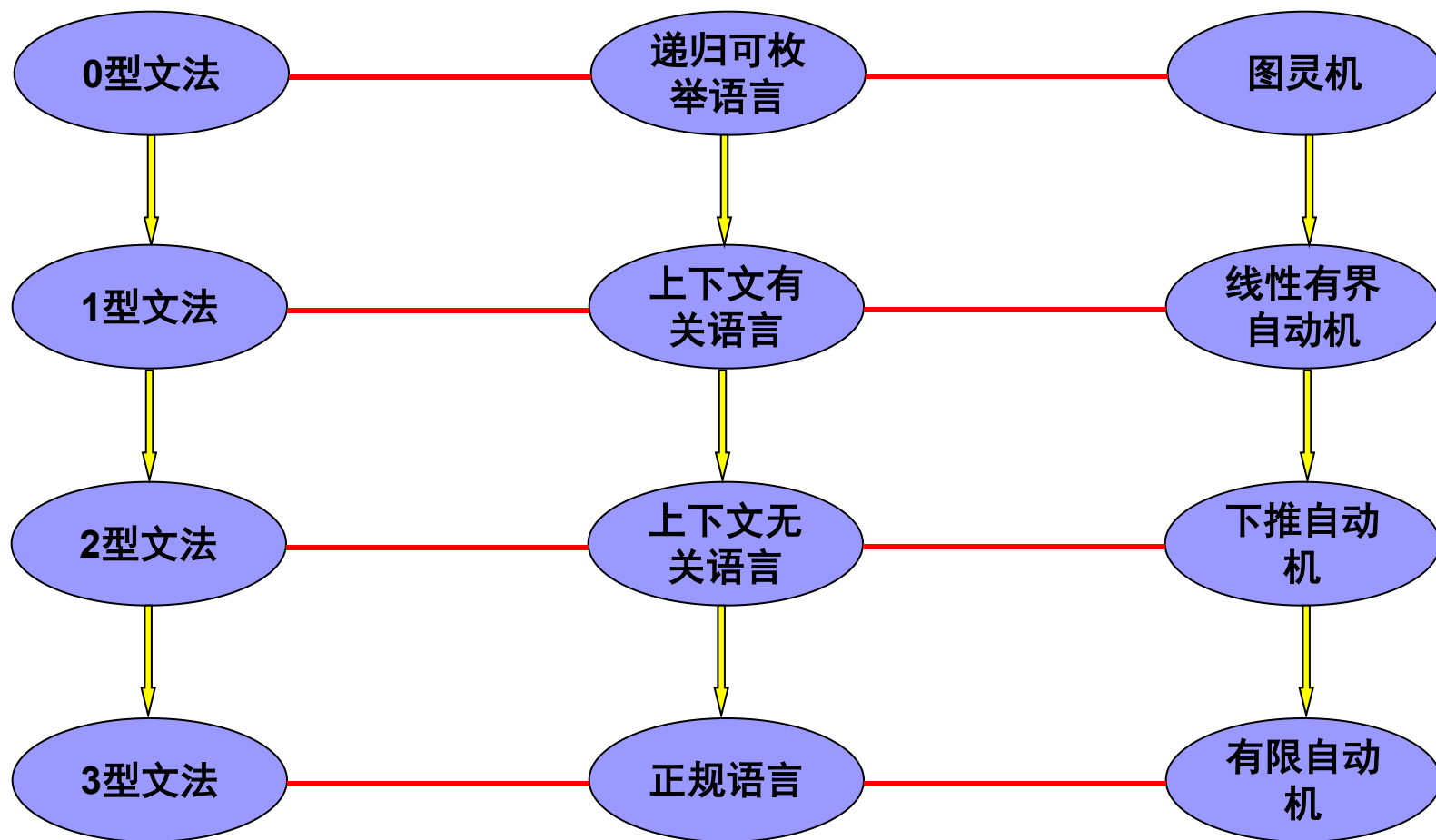
$cC \rightarrow cc$

$cC \rightarrow c$

Chomsky 文法体系



文法、形式语言和自动机的对应关系



总结 (1)

- 自然语言是上下文有关的。
- 但是上下文无关文法有足够的描述能力描述现今多数程序设计语言的语法结构
 - 算术表达式
 - 语句
 - 赋值语句
 - 条件语句
 - 读语句

上下文无关文法描述语言语法结构

- 算术表达式 的上下文无关文法表示

文法 $G = (\{E\}, \{+, *, i, (,)\}, P, E)$

$P: E \rightarrow i \quad E \rightarrow E + E$

$E \rightarrow E * E \quad E \rightarrow (E)$

- 条件语句的上下文无关文法表示

$\langle \text{条件语句} \rangle \rightarrow \text{if} \langle \text{条件表达式} \rangle \text{then} \langle \text{语句} \rangle$

$\quad \quad \quad | \text{if} \langle \text{条件表达式} \rangle \text{then} \langle \text{语句} \rangle \text{else} \langle \text{语句} \rangle$

总结 (2)

- 基于正规文法讨论词法分析问题，
- 基于上下文无关文法讨论语法分析问题。
- 在编译过程中，一般用上下文无关文法和正则文法描述词法语法，而其中上下文有关的问题，可通过表格处理解决。

例 非CFL的文法

$L = \{a^n b^n c^n | n > 0\}$ 的文法

$S \rightarrow aBC | aSBC$

$CB \rightarrow BC$

$aB \rightarrow ab$

$bB \rightarrow bb$

$bC \rightarrow bc$

- 可以证明不存在CFG G , 使 $L(G) = L$

非CFL结构

- 在我们使用的程序语言中,有些语言结构并不是总能用上下文无关文法描述的。
 - **例1** $L1=\{wcw|w\in\{a,b\}^+\}$ 。aabcaab就是L1的一个句子。这个语言是检查程序中标识符的声明应先于引用的抽象。
 - **例2** $L2=\{a^n b^m c^n d^m | n, m \geq 0\}$, 它是检查过程声明的形参个数和过程引用的参数个数是否一致问题的抽象。

语法分析树和文法的二义性

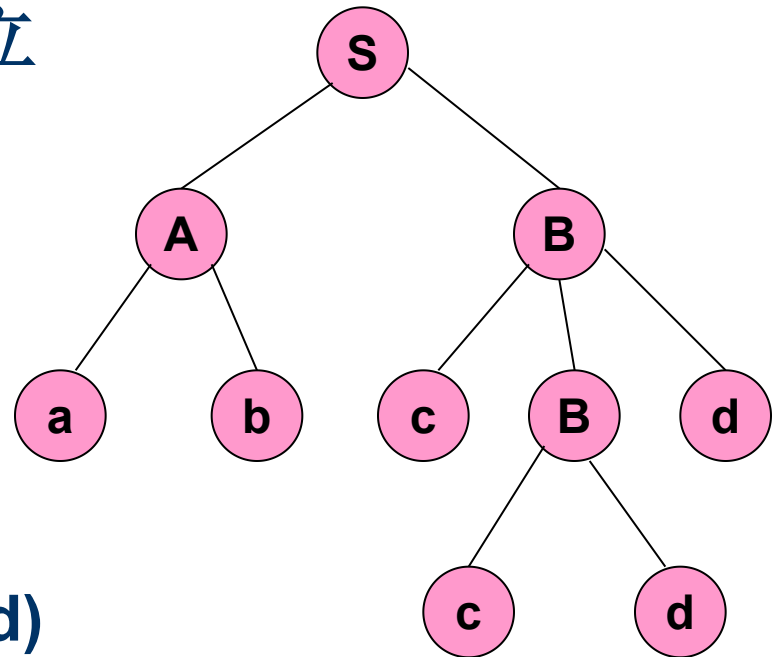
- 语法分析树，简称语法树
- 二义性



什么是语法树？

■ 语法树是表示一个句型推导过程的图，是一棵倒立的树。

- 结点
- 边
- 根结点
- 末端结点
- 末端分支：A(a,b)和B(c,d)
- 兄弟结点



从推导构造语法树

- 方法：把开始符号S做为语法树的根结点，对每一个直接推导画一次结点的扩展，该结点是直接推导中被替换的非终结符号，其所有儿子结点所组成的符号串是推导中所用产生式右部。直到推出或句型或句子或无法再推导时结束。

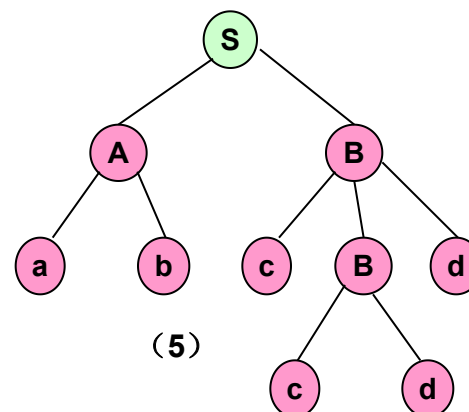
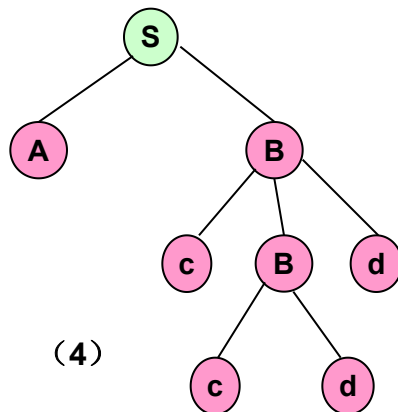
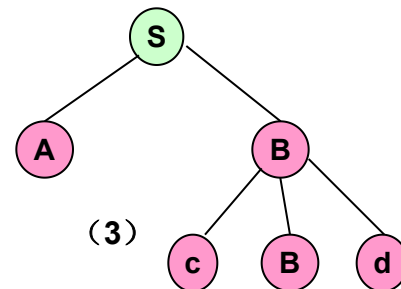
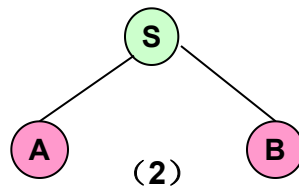
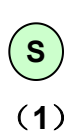
语法树的构造过程

$S \Rightarrow AB$

$\Rightarrow AcBd$

$\Rightarrow Accdd$

$\Rightarrow abccdd$



语法树的特征

- 给定文法**G**， $G=(V_N, V_T, S, \mathcal{L})$ ，对于**G**的任何句型都能构造与之关联的语法树（推导树）。这棵树具有下列特征：
 - 1、根结点的标记是开始符号**S**；
 - 2、每个结点的标记都是**V**中的一个符号；
 - 3、若一棵子树的根结点为**A**，且其所有儿子结点的标记从左向右的排列为**A₁A₂...A_R**，那么 $A \rightarrow A_1A_2..A_R$ 一定是 $\mathcal{L}$ 中的一条产生式（规则）；
 - 4、若一标记为**A**的结点至少有一个儿子，则 $A \in V_N$
 - 5、树的叶结点符号所组成的符号串**w**就是所给句型；若**w**中仅含终结符号，则**w**为文法**G**所产生的句子。

■例. 文法 $G[E]$:

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid i$

句型 $E+F*i$ 推导:

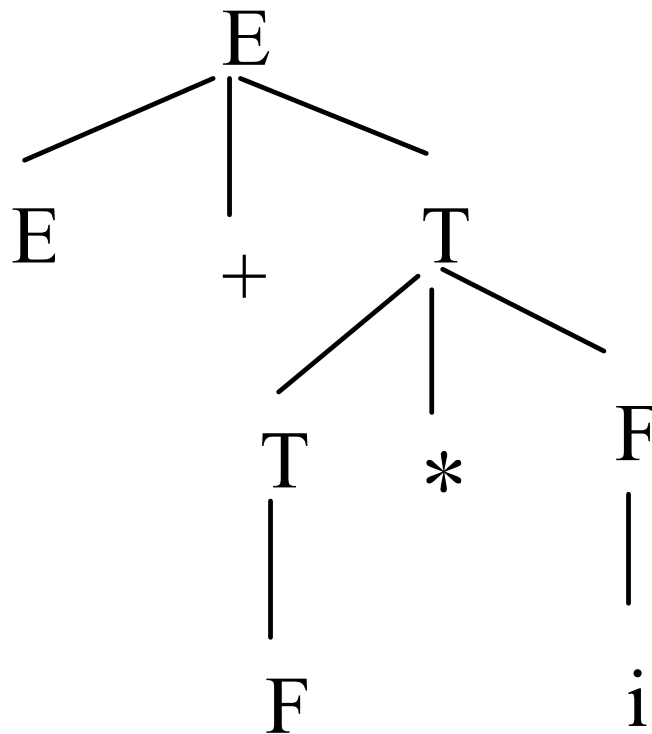
$E \Rightarrow E+T$

$\Rightarrow E+T * F$

$\Rightarrow E+F * F$

$\Rightarrow E+F * i$

对应的语法树如右图



语法树解释

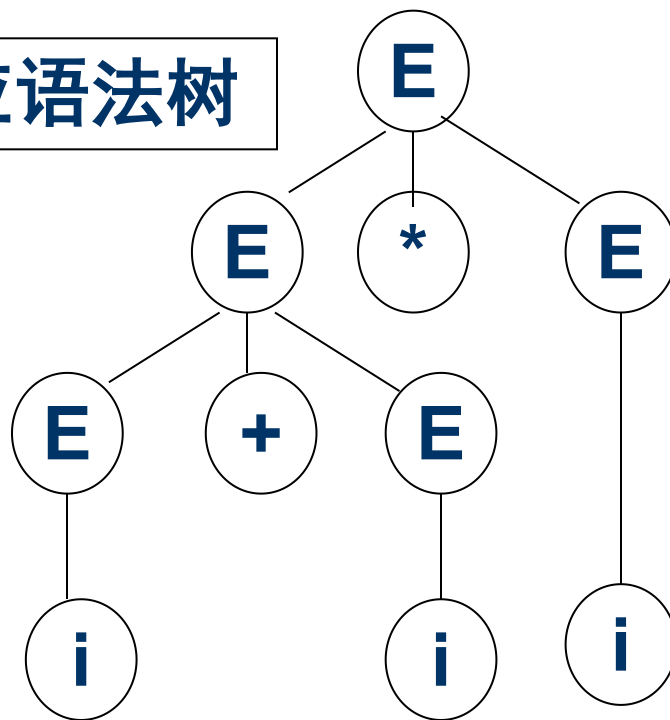
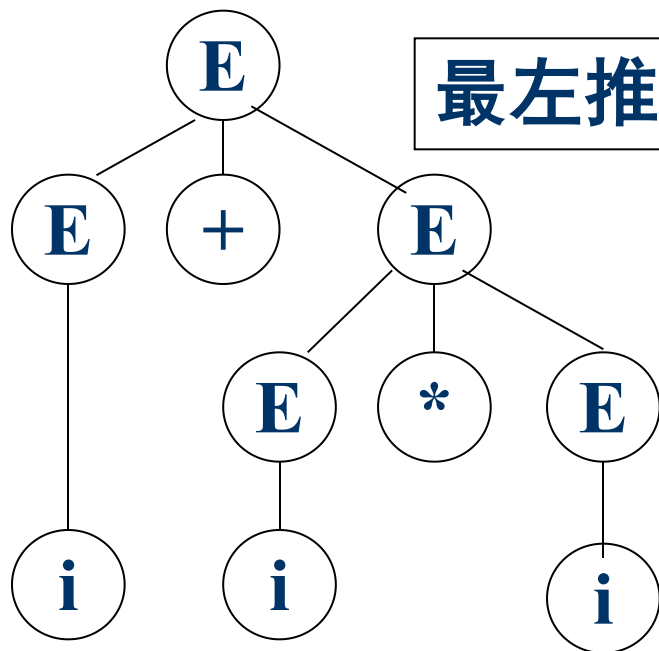
- 语法树表明了推导过程中使用了哪条规则和使用在哪个非终结符号上，但它并没有表明使用规则（产生式）的顺序。
- 一个句型是否只有唯一的一个最左(最右)推导呢？
 - 否！
- 一个句型是否只对应唯一的一棵语法树呢？
 - 否！
 - 因为不同的最右（左）推导对应不同的语法树。

文法G[E]: $E \rightarrow E + E \mid E * E \mid (E) \mid i$

$E \Rightarrow E + E \Rightarrow i + E$
 $\Rightarrow i + E * E \Rightarrow i + i * E$
 $\Rightarrow i + i * i$

$E \Rightarrow E * E \Rightarrow E + E * E$
 $\Rightarrow i + E * E \Rightarrow i + i * E$
 $\Rightarrow i + i * i$

最左推导及相应语法树



同理，该句子的最右推导及相应语法树也不同

二义性

- 如果一个文法的句子（句型）存在两棵语法树,那么,该句子是二义性的。
- 如果一个文法包含二义性的句子,则称这个文法是二义性的; 否则,该文法是无二义性的。

说明

1. 一般来说，程序语言存在无二义性的文法描述，但是对于条件语句，经常使用二义性文法描述它：

$S \rightarrow \text{if expr then } S$
 $\quad \quad \quad | \text{if expr then } S \text{ else } S \mid \text{other}$

二义性的句子：

$\text{if } e1 \text{ then if } e2 \text{ then } s1 \text{ else } s2$

可以用无二义文法来描述，但是比较复杂

2. 在能驾驭的情况下，使用二义性文法。

语言的二义性与文法的二义性问题

- 如语言L找到一个文法是无二义的，则L是无二义的；
- 如未找到一个文法是无二义的，也不能断定L二义。
- 产生某上下文无关语言的每一个文法都是二义的，则称此语言是先天二义的。

例. $\{a^i b^j c^j \mid i, j \geq 1\} \cup \{a^i b^j c^i \mid i, j \geq 1\}$

存在一个二义性的句子 $a^k b^k c^k$

- 文法的二义性是不可判定的。
(因为文法的二义性由句子的语法树决定，不可能对无穷句子来判别)

二义文法改造为无二义文法

- 文法 $G[E]$: $E \rightarrow E+E \mid E \times E \mid (E) \mid i$ 是二义性的, 如果规定 “ $\times$ ” 和 “ $+$ ” 的优先性, 并服从左结合, 上式就可以构造出无二义性文法。

文法 $G[E]$:

$$E \rightarrow T \mid E \times T$$
$$T \rightarrow F \mid T + F$$
$$F \rightarrow (E) \mid i$$

句子的推导过程唯一: 如: $i \times i + i$

作业

■ P36

6, 8, 9, 10